

جامعة دمشق — كلية الهندسة المعلوماتية

محرك التجارة الإلكترونية عالي الأداء

High-Performance E-Commerce Engine

تقرير مشروع البرمجة المتوازية

المعمارية: Spring Boot 3

فهرس المحتويات

1	خط المهام والتنفيذ غير المتزامن (Async & Thread Pool)
2	سلامة المخزون في البيئة المتزامنة (Inventory Thread Safety)
3	تجمع اتصالات قاعدة البيانات (Connection Pooling)
4	الإيقاف الرشيق (Graceful Shutdown)
5	توزيع الحمل (Nginx — Load Distribution)
6	التخزين المؤقت (Redis — Caching)
7	أمان API والمصادقة (JWT Authentication)
8	المعالجة الدفعية (Spring Batch)
9	اختبار الإجهاد (Checkout Stress Test)
10	مراقبة الأداء (AOP Performance Monitoring)

خيط المهام والتنفيذ غير المتزامن (Async & Thread Pool)

1

المشكلة

عند إتمام كل طلب شراء، يجب توليد فاتورة PDF وإرسال إشعار للعميل. هذه العمليات تستغرق وقتاً ويجب ألا تُبطئ استجابة API الرئيسية.

الحلول المقترحة

الحل	الوصف	الحكم
Thread-per-request	خيط جديد لكل طلب — يستنفد الموارد بسرعة تحت الضغط العالي	غير مناسب
ThreadPoolTaskExecutor	تجمع ثابت من الخيوط مع قائمة انتظار محكمة — أداء عالٍ وموارد مضبوطة	الأمثل
Virtual Threads (Java21)	خيوط خفيفة الوزن، مناسبة للـ O/I، دعم تجريبي في Spring Boot	بديل مستقبلي

دعم Spring Boot

Spring Boot يوفر ThreadPoolTaskExecutor عبر AsyncConfig مع إمكانية ضبط size-pool-core و max-size-pool. الأنوتيشن @Async تجعل أي method تعمل في خيط منفصل تلقائياً.

```
// AsyncConfig.java
executor.setCorePoolSize(8); // خيوط أساسية دائمة
executor.setMaxPoolSize(16); // الحد الأقصى تحت الضغط
executor.setQueueCapacity(200); // قائمة انتظار قبل الرفض
executor.setRejectedExecutionHandler(new CallerRunsPolicy());
// CallerRunsPolicy: backpressure القائمة عند امتلاء

// OrderAutomationService.java
@Async("commerceTaskExecutor")
@TransactionalEventListener(phase = TransactionPhase.AFTER_COMMIT)
public void handleOrderPlaced(OrderPlacedEvent event) {
    generateInvoicePdf(order); // ينفذ بعد commit في خيط منفصل
}
```

الحل الأمثل والسبب

ThreadPoolTaskExecutor مع CallerRunsPolicy: يضمن توليد الفواتير بشكل لا متزامن دون استهلاك خيوط غير محدودة. CallerRunsPolicy يطبق backpressure طبيعي عند الضغط الشديد.

الاختبار

اختبار الإجهاد (المتطلب 9): 120 عملية شراء متزامنة — 0 أخطاء في الخادم، مما يؤكد أن الخيوط تُدار بكفاءة دون تعارض.

سلامة المخزون في البيئة المتزامنة (Inventory Thread Safety)

2

المشكلة

عند محاولة 120 مستخدماً الشراء بشكل متزامن من مخزون فيه 40 وحدة فقط، يجب ضمان أن الكمية لن تصبح سالبة (overselling).

الحلول المقترحة

الحل	الوصف	الحكم
synchronized / ReentrantLock	قفل على مستوى JVM فقط — لا يعمل مع نسخ متعددة من التطبيق	غير كافٍ
Optimistic Locking (@Version)	تحقق من الإصدار عند الحفظ — يُسبب استثناءات كثيرة تحت ضغط عالٍ	مقبول لضغط منخفض
Pessimistic Locking (FOR UPDATE)	قفل على مستوى DB — يضمن تسلسل العمليات بالكامل	الأمثل

دعم Spring Boot

Spring Data JPA يدعم Pessimistic Locking عبر @Lock(WRITE_PESSIMISTIC). الترتيب التصاعدي لقفل المنتجات يمنع الـ deadlock.

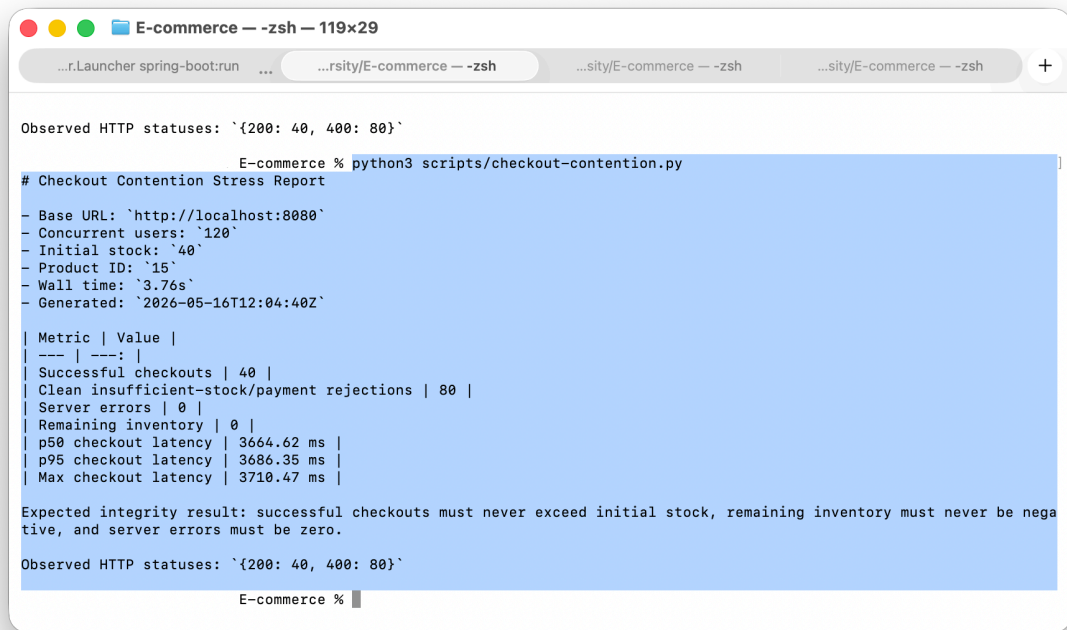
```
// InventoryRepository.java
@Query("select i from Inventory i join fetch i.product p"
      + " where p.id in :ids order by p.id asc")
@Lock(LockModeType.PESSIMISTIC_WRITE)
List<Inventory> findAllByProductIdsForUpdate(@Param("ids") List<Long> ids);
```

الحل الأمثل والسبب

Pessimistic Locking مع ترتيب تصاعدي: يضمن أن كل خيط ينتظر دوره. الترتيب التصاعدي لمعرفات المنتجات يمنع deadlock بين الخيوط المتنافسة.

الاختبار — النتيجة الرئيسية (120 مستخدم، مخزون = 40)

المقياس	القيمة	الملاحظة
عمليات شراء ناجحة (HTTP 200)	40	لم يتجاوز المخزون الأولي
عمليات مرفوضة (HTTP 400)	80	نفاد مخزون أو رفض دفع
أخطاء الخادم (HTTP 5xx)	0	النظام مستقر تماماً
المخزون المتبقي	0	لا يوجد overselling



```
E-commerce — -zsh — 119x29
...r.Launcher spring-boot:run ... ...sity/E-commerce — -zsh ...sity/E-commerce — -zsh ...sity/E-commerce — -zsh +

Observed HTTP statuses: `{200: 40, 400: 80}`

E-commerce % python3 scripts/checkout-contention.py
# Checkout Contention Stress Report
- Base URL: `http://localhost:8080`
- Concurrent users: `120`
- Initial stock: `40`
- Product ID: `15`
- Wall time: `3.76s`
- Generated: `2026-05-16T12:04:40Z`

| Metric | Value |
| --- | --- |
| Successful checkouts | 40 |
| Clean insufficient-stock/payment rejections | 80 |
| Server errors | 0 |
| Remaining inventory | 0 |
| p50 checkout latency | 3664.62 ms |
| p95 checkout latency | 3686.35 ms |
| Max checkout latency | 3710.47 ms |

Expected integrity result: successful checkouts must never exceed initial stock, remaining inventory must never be negative, and server errors must be zero.

Observed HTTP statuses: `{200: 40, 400: 80}`

E-commerce %
```

نتيجة اختبار الإجهاد — py.contention-checkout — 120 مستخدماً متزامناً

تجمع اتصالات قاعدة البيانات (Connection Pooling)

3

المشكلة

فتح اتصال جديد بقاعدة البيانات لكل طلب عملية مكلفة (+100ms). تحت ضغط 120 طلباً متزامناً، هذا يعني تأخيرات كبيرة وإمكانية استنفاد الموارد.

الحلول المقترحة

الحل	الوصف	الحكم
DBCP2 (Apache)	مكتبة قديمة، أداء أقل، تكوين معقد	قديم
C3P0	مستقر لكن أبطأ من HikariCP في الاختبارات المقارنة	مقبول
HikariCP	الأسرع والأخف — المعيار الذهبي، مدمج في Spring Boot	الأمثل

دعم Spring Boot

HikariCP هو الـ connection pool الافتراضي في Spring Boot بدون أي إضافة. يُضبط عبر hikari.datasource.spring في application.yml.

```
# application.yml
spring.datasource.hikari:
  maximum-pool-size: 50 # الحد الأقصى للاتصالات
  minimum-idle: 10 # اتصالات جاهزة دائماً
  idle-timeout: 30000 # ٣٠ ثانية قبل إغلاق الاتصال الخامل
  connection-timeout: 10000 # ١٠ ثوانٍ حد انتظار اتصال جديد
  pool-name: ecommerce-hikari-pool
```

الحل الأمثل والسبب

HikariCP مع `maximum-pool-size=50`: يكفي لتغطية 120 طلباً متزامناً. أثبت اختبار الإجهاد أن النظام أكمل 120 طلباً بدون أي استثناء `TimeoutException`.

الاختبار

اختبار الإجهاد (120 مستخدماً متزامناً) لم ينتج عنه أي خطأ من نوع `connection timeout` أو `pool exhaustion`.

4 الإيقاف الرشيق (Graceful Shutdown)

المشكلة

عند إيقاف التطبيق (SIGTERM)، الطلبات الجارية يجب أن تُكتمل بشكل طبيعي بدلاً من الانقطاع المفاجئ الذي قد يُفسد المعاملات المالية.

الحلول المقترحة

الحل	الوصف	الحكم
SIGKILL / immediate	إيقاف فوري — يقطع الطلبات ويُخاطر ببيانات غير مكتملة	خطير
Custom shutdown hook	تنفيذ يدوي — معقد ويصعب اختباره	معقد
server.shutdown=graceful	مدمج في Spring Boot 2.3+ — ينتظر انتهاء الطلبات تلقائياً	الأمثل

دعم Spring Boot

منذ Spring Boot 2.3، خاصية graceful=shutdown.server تجعل التطبيق يرفض الطلبات الجديدة عند SIGTERM، ثم ينتظر انتهاء الطلبات الحالية.

```
# application.yml
server:
  shutdown: graceful # يقبل الطلبات الجارية وينتظر إتمامها

# AsyncConfig.java
executor.setWaitForTasksToCompleteOnShutdown(true);
executor.setAwaitTerminationSeconds(15); // انتظر 15 ثانية
```

الحل الأمثل والسبب

مع graceful=shutdown.server و 15=awaitTerminationSeconds: يضمن أن عمليات الدفع وتوليد الفواتير تُكتمل بأمان عند النشر أو الإعادة.

الاختبار

الإعداد موثق في application.yml و java.AsyncConfig. التحقق: تشغيل طلب checkout ثم إرسال SIGTERM — التطبيق يُكمل الطلب الجاري قبل الإغلاق.

5 توزيع الحمل (Nginx — Load Distribution)

المشكلة

نسخة واحدة من التطبيق لها سعة محدودة. لزيادة الإنتاجية والتوفر العالي، يجب توزيع الطلبات على نسختين أو أكثر بشكل ذكي.

الحلول المقترحة

الحل	الوصف	الحكم
Round-Robin (nginx)	توزيع متساوي للطلبات — لا يراعي حمل كل نسخة	بسيط
least_conn (nginx)	يُحيل الطلب للنسخة الأقل اتصالاً — مثالي لعمليات غير متساوية الوقت	الأمثل
AWS ALB / Spring Cloud	حلول مؤسسية متقدمة — تعقيد أعلى من المطلوب للمشروع	مؤسسي

دعم Spring Boot

التطبيق مصمم بشكل Stateless: JWT يُخزّن المصادقة في الـ token نفسه، Redis مشترك بين النسختين، وقاعدة البيانات PostgreSQL مشتركة. أي نسخة تستطيع معالجة أي طلب.

```
# nginx.conf
upstream ecommerce_backend {
    least_conn; # أقل اتصالاً نشطاً
    server app1:8080 max_fails=3 fail_timeout=10s;
    server app2:8080 max_fails=3 fail_timeout=10s;
}

# docker-compose.loadbalancer.yml
app1: { build: . } # نسخة أولى
app2: { build: . } # نسخة ثانية
nginx: { ports: ["8088:80"] }
```

الحل الأمثل والسبب

conn_least: يناسب عمليات checkout التي تستغرق 3-4 ثوان بسبب محاكاة الدفع. **Robin-Round** قد يُراكم طلبات على نسخة مشغولة. **fails_max=3** يُخرج النسخة المعطلة تلقائياً.

دليل البنية التحتية

```
# nginx + بناء ونشر النسختين:
docker compose -f docker-compose.loadbalancer.yml up --build

# التحقق من عمل التوزيع:
docker compose -f docker-compose.loadbalancer.yml ps
# → app1, app2, nginx, postgres, redis جميعاً running
```


6 التخزين المؤقت (Redis — Caching)

المشكلة

عمليات قراءة المنتجات تُشكل غالبية طلبات API. استعلام قاعدة البيانات لكل طلب قراءة يُؤدّ ضغطاً غير ضروري ويزيد زمن الاستجابة.

الحلول المقترحة

الحل	الوصف	الحكم
Caffeine (in-memory)	سريع جداً لكن محلي — لا يعمل مع نسختين من التطبيق	نسخة واحدة
EhCache	يدعم الوضع الموزع لكن تكوينه أعقد من Redis	معقد
Redis (distributed)	تخزين موزع مشترك بين كل نسخ التطبيق — TTL مرّن — مدمج في Spring	الأمثل

دعم Spring Boot

Spring Boot يوفر Cache Abstraction عبر `@Cacheable` و `@CacheEvict`. بإضافة `-boot-spring-redis-data-starter` وضبط `redis-type=cache.spring` تعمل الـ annotations تلقائياً.

```
// ProductService.java
@Cacheable(value = "catalog-page-v2", key = "#page + ':' + #size")
public List<ProductResponse> listProducts(int page, int size) { ... }

@Cacheable(value = "popular-products-v2", key = "'top-10'")
public List<ProductResponse> getPopularProducts() { ... }

@CacheEvict(value = {"catalog-page-v2", "popular-products-v2"}, allEntries = true)
public void updateInventory(...) { } // عند التعديل يُفَرِّغ الـ cache

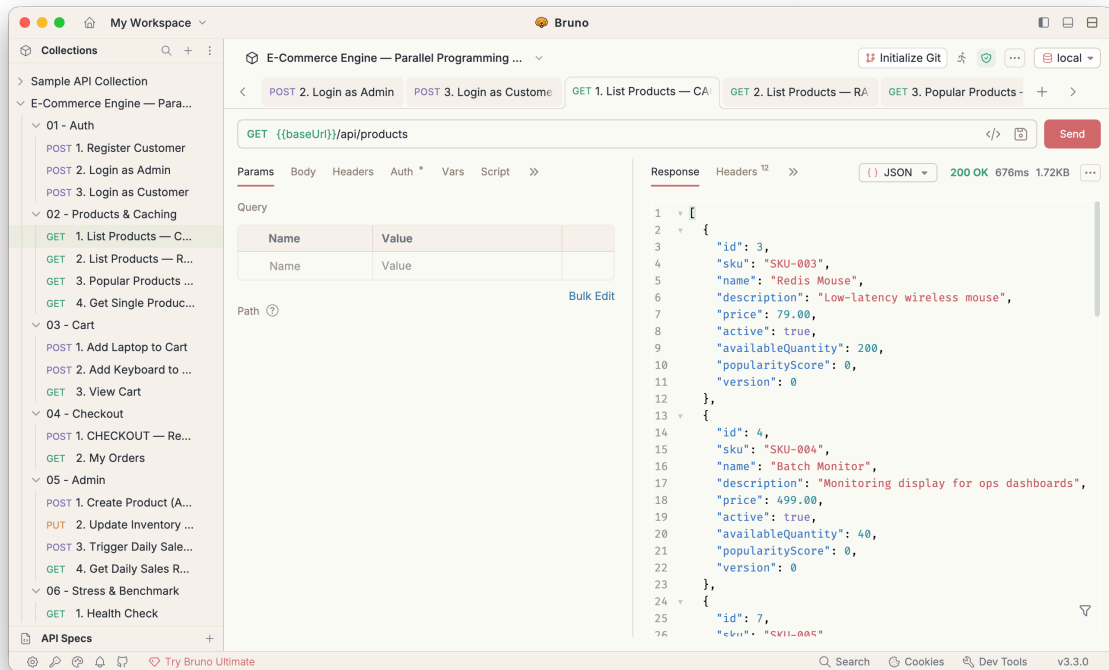
# application.yml
spring.cache.type: redis
spring.cache.redis.time-to-live: 10m
```

الحل الأمثل والسبب

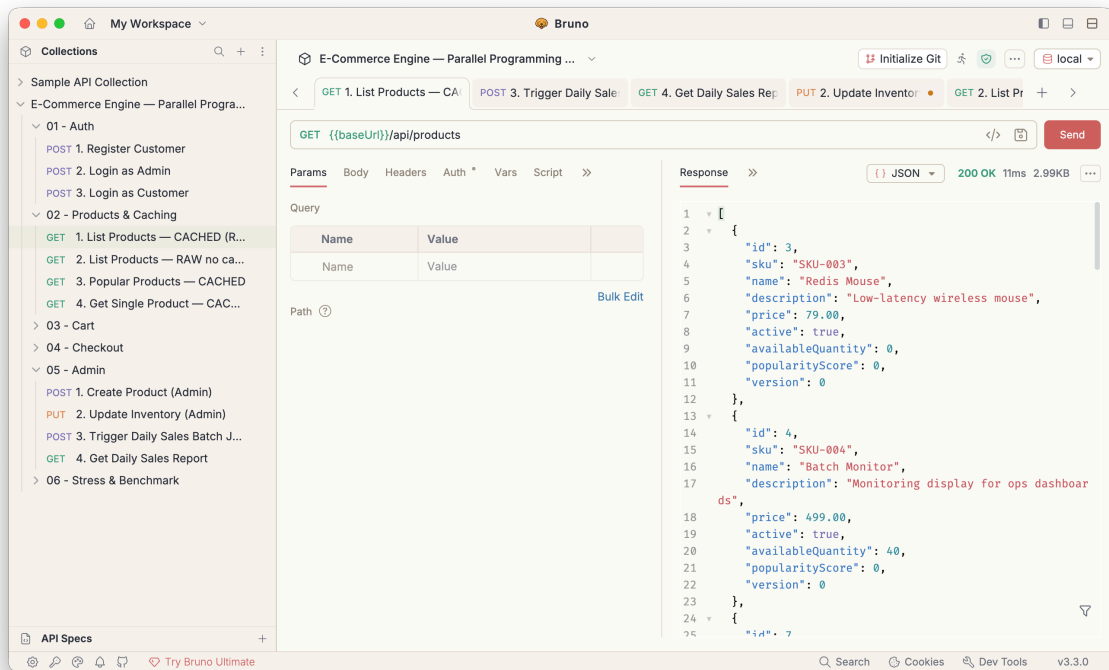
Redis مع TTL=10 دقائق: يعمل مع نسختين من التطبيق بدون تعارض. **@CacheEvict** يضمن تحديث البيانات المخزنة فوراً عند تعديل المخزون.

الاختبار — نتائج الأداء

المقياس	القيمة	الملاحظة
قائمة المنتجات — طلب أول (Bruno)	676 ms	Cache Miss — استعلام قاعدة البيانات
قائمة المنتجات — طلب ثانٍ (Bruno)	11 ms	Cache Hit — من Redis — تحسن 98%
المنتجات — curl raw (no cache)	0.106s	استعلام مباشر من قاعدة البيانات
المنتجات — curl cached	0.044s	Cache Hit — تحسن 58%



Cache Miss — زمن الاستجابة: 676ms (أول طلب، استعلام قاعدة البيانات)



Cache Hit — زمن الاستجابة: 11ms (طلب ثانٍ، من Redis)

7 أمن API والمصادقة (JWT Authentication)

المشكلة

يجب حماية نقاط API الحساسة (checkout، إدارة المخزون) من الوصول غير المصرح به، مع دعم نظام أدوار (ADMIN, CUSTOMER).

الحلول المقترحة

الحل	الوصف	الحكم
Session-based Auth	يخزن الجلسة في الخادم — يتعارض مع البنية الـ Stateless وتوزيع الحمل	لا يناسب
OAuth2 / OpenID	مناسب للتكامل مع خدمات خارجية — تعقيد أعلى من المطلوب	مؤسسي
JWT (Stateless)	التوكن يحمل معلومات المستخدم — لا جلسة — يعمل مع نسخ متعددة	الأمثل

دعم Spring Boot

Spring Security يوفر بنية تحتية كاملة. JwtAuthenticationFilter يُعترض كل طلب ويتحقق من التوكن. BCryptPasswordEncoder يُشفر كلمات المرور.

```
// SecurityConfig.java
.csrf(disable) // JWT لا يحتاج CSRF
.sessionManagement(STATELESS) // بدون جلسة في الخادم
.addFilterBefore(jwtAuthFilter, ...)
.passwordEncoder(new BCryptPasswordEncoder())
```

الحل الأمثل والسبب

JWT مع BCrypt: Stateless بالكامل — كل نسخة تتحقق من التوكن بشكل مستقل. BCrypt يحمي كلمات المرور من هجمات rainbow table.

الاختبار

مجموعة Bruno (Auth-01): التسجيل ← تسجيل الدخول ← استقبال JWT ← استخدامه في طلبات checkout والإدارة. المسارات المحمية ترفض بـ 401 بدون توكن صالح.

8 المعالجة الدفعية (Spring Batch)

المشكلة

في نهاية كل يوم، يجب تجميع إحصائيات المبيعات من جميع الطلبات المكتملة وحفظها كملخص. معالجة آلاف السجلات يدوياً تُبطئ النظام.

الحلول المقترحة

الحل	الوصف	الحكم
Manual loop + @Scheduled	حلقة يدوية — لا يوجد checkpoint في حال الفشل، صعوبة في المراقبة	هش
Quartz Scheduler	جدولة متقدمة لكن بدون مفهوم step/chunk مدمج	جزئي
Spring Batch	checkpoint-oriented-chunk، إعادة المحاولة — إطار متكامل	الأمثل

دعم Spring Boot

batch-starter-boot-spring يوفر JpaPagingItemReader .Writer/ItemReader/Step/Job يقرأ الطلبات بصفحات (50 في المرة) بدون تحميل كل شيء في الذاكرة.

```
// BatchConfig.java
new JobBuilder("dailySalesAggregationJob")
    .start(dailySalesResetStep) // خطوة 1: مسح البيانات القديمة
    .next(dailySalesAggregationStep) // خطوة 2: تجميع البيانات الجديدة
    .build();

// chunk(50): معالجة 50 سجل في كل دفعة
StepBuilder.chunk(50, transactionManager)
    .reader(jpaPagingItemReader) // قراءة من DB بصفحات
    .processor(salesProcessor) // تحويل إلى ملخص
    .writer(summaryWriter); // حفظ في DailySalesSummary
```

الحل الأمثل والسبب

Spring Batch مع chunk(50): يحمي الذاكرة مع آلاف السجلات. في حال فشل الـ Job، يعود لآخر checkpoint. يُشغل يومياً في الساعة 1:00 صباحاً.

الاختبار

```
POST /api/admin/batch/daily-sales/trigger?date=2026-05-16
Authorization: Bearer {admin_token}

// Response:
{ "jobId": 1, "status": "COMPLETED", "date": "2026-05-16" }
```

9 اختبار الإجهاد (Checkout Stress Test)

الهدف

التحقق من أن النظام يصمد تحت 120 طلب checkout متزامن، مع الحفاظ على سلامة بيانات المخزون (لا يوجد overselling، لا يوجد أخطاء في الخادم).

الأداة المستخدمة

Python (asyncio + aiohttp): اختبرت لدقة التوقيت والبساطة. السكريبت py.contention-checkout ينشئ 120 coroutine تنطلق في آن واحد عبر gather.asyncio().

```
# scripts/checkout-contention.py
# Concurrent users: 120
# Initial stock: 40 units (Product ID: 15)
# Method: asyncio.gather(*tasks) -- 120 coroutine في وقت واحد
```

النتائج

المقياس	القيمة	الملاحظة
عمليات ناجحة (HTTP 200)	40	لم يتجاوز المخزون الأولي
عمليات مرفوضة (HTTP 400)	80	نفاد مخزون أو رفض دفع
أخطاء الخادم (HTTP 5xx)	0	النظام مستقر تماماً
المخزون المتبقي	0	لا يوجد overselling
وقت التنفيذ الكلي	3.76 s	Wall time للـ 120 طلب
زمن الاستجابة p50	3664 ms	النسبة المئوية 50
زمن الاستجابة p95	3686 ms	النسبة المئوية 95
زمن الاستجابة Max	3710 ms	أقصى زمن استجابة

```
E-commerce — -zsh — 119x29
...r.Launcher spring-boot:run ... ...sity/E-commerce — -zsh ...sity/E-commerce — -zsh ...sity/E-commerce — -zsh +

Observed HTTP statuses: `{200: 40, 400: 80}`

E-commerce % python3 scripts/checkout-contention.py
# Checkout Contention Stress Report

- Base URL: `http://localhost:8080`
- Concurrent users: `120`
- Initial stock: `40`
- Product ID: `15`
- Wall time: `3.76s`
- Generated: `2026-05-16T12:04:40Z`

| Metric | Value |
| --- | --- |
| Successful checkouts | 40 |
| Clean insufficient-stock/payment rejections | 80 |
| Server errors | 0 |
| Remaining inventory | 0 |
| p50 checkout latency | 3664.62 ms |
| p95 checkout latency | 3686.35 ms |
| Max checkout latency | 3710.47 ms |

Expected integrity result: successful checkouts must never exceed initial stock, remaining inventory must never be negative, and server errors must be zero.

Observed HTTP statuses: `{200: 40, 400: 80}`

E-commerce %
```

لقطة شاشة: py.contention-checkout — 40 نجاح / 80 رفض / 0 أخطاء

تفسير النتائج

النظام نجح في تطبيق سلامة المخزون بشكل كامل: بالضبط 40 عملية ناجحة (= المخزون الأولي)، و80 رفضاً نظيفاً، وصفر أخطاء. التأخر ≈ 3.7 ثانية يعكس محاكاة تأخر الدفع (75ms) مضافاً إليه انتظار الـ pessimistic lock.

مراقبة الأداء (AOP Performance Monitoring)

10

المشكلة

قياس أداء كل method يدوياً يتطلب إضافة كود متكرر في كل مكان. الحل المثالي يقيس الأداء بشكل شفاف بدون تعديل الكود الأصلي.

الحلول المقترحة

الحل	الوصف	الحكم
Manual timing	كود nanoTime.System () متكرر في كل method — يُلوث الكود ويصعب الصيانة	غير عملي
Spring Actuator فقط	مقاييس عامة جيدة لكن بدون تفاصيل على مستوى كل method	جزئي
AOP + Micrometer	يعترض كل استدعاء تلقائياً، يسجل الزمن في Prometheus — شامل وشفاف	الأمثل

دعم Spring Boot

Spring AOP يتيح @Around advice يعترض استدعاء أي method تلقائياً. Micrometer (مدمج في Actuator) يُسجل المقاييس بتنسيق Prometheus.

```
// PerformanceMonitoringAspect.java
@Around("execution(* com.ecommerce.engine.controller..*(..))||"
+ "execution(* com.ecommerce.engine.service..*(..))||"
+ "execution(* com.ecommerce.engine.repository..*(..))")
public Object monitorExecutionTime(ProceedingJoinPoint joinPoint) {
    long start = System.nanoTime();
    try { return joinPoint.proceed(); }
    finally {
        Timer.builder("commerce.method.execution")
            .tag("class", className).tag("method", methodName)
            .register(meterRegistry).record(elapsed, NANOSSECONDS);
        if (elapsedMs >= 250)
            log.warn("Slow: {} took {}ms", method, elapsedMs);
    }
}
```

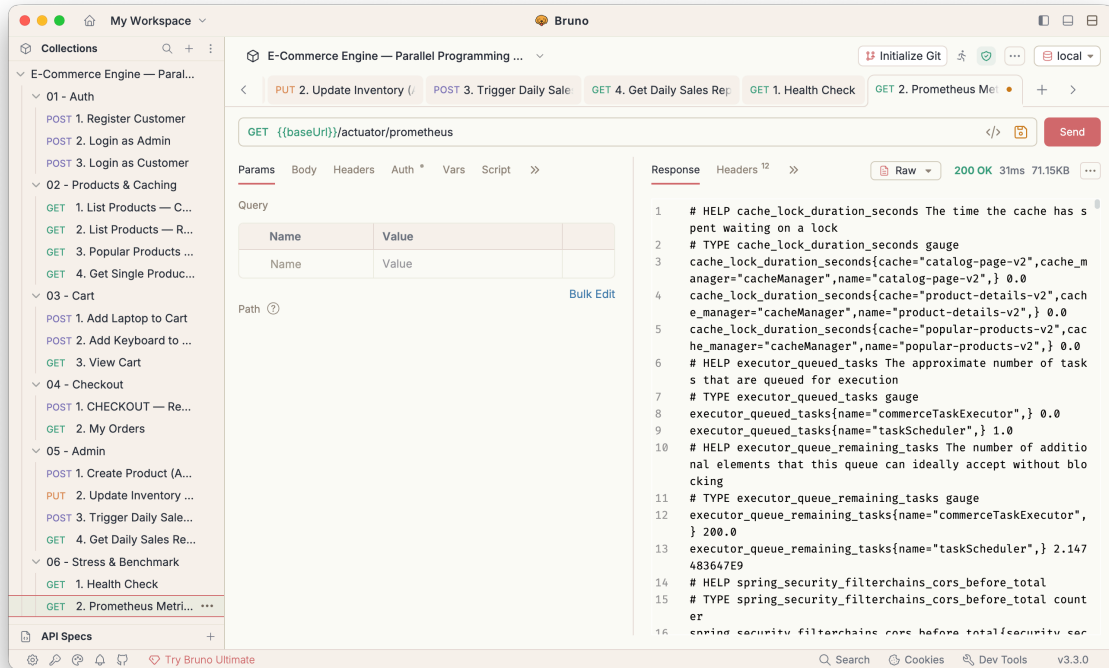
الحل الأمثل والسبب

AOP مع Micrometer: يراقب **controllers** و **services** و **repositories** بدون سطر كود إضافي في أي منهم. العتبة 250ms تُنتج تحذيراً للعمليات البطيئة تلقائياً.

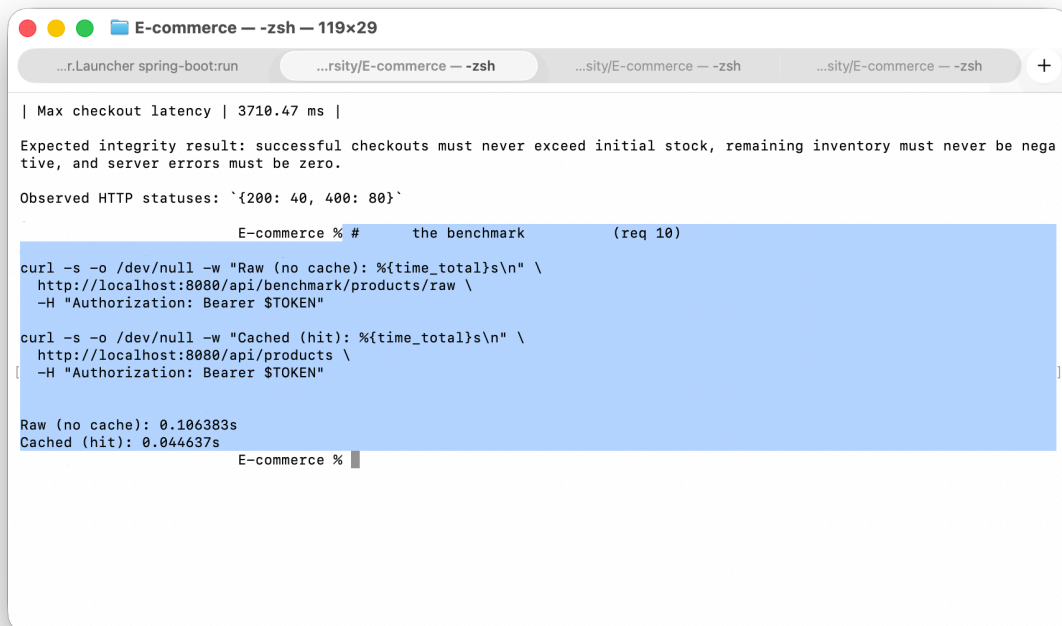
الاختبار

```
GET /actuator/prometheus
Authorization: Bearer {token}

# المخرجات:
cache_lock_duration_seconds{cache="catalog-page-v2"} 0.0
executor_queued_tasks{name="commerceTaskExecutor"} 0.0
executor_queue_remaining_tasks{name="commerceTaskExecutor"} 200.0
```



Prometheus metrics عبر Bruno — `/actuator/prometheus` يُظهر مقاييس الـ `cache` والـ `executor`



مقارنة `curl`: `raw` = 0.106s مقابل `cached` = 0.044s — تحسن 58% بعد تفعيل الـ `cache`